

# OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

## Module 3 Exam Notes: Classes, Inheritance & Polymorphism

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Class Fundamentals and Objects</b>               | <b>2</b> |
| 1.1      | Constructors in Java                                | 2        |
| 1.2      | Argument Passing in Java                            | 2        |
| 1.3      | The <code>static</code> Keyword                     | 3        |
| 1.4      | Nested and Inner Classes                            | 3        |
| <b>2</b> | <b>Inheritance and Reuse Models</b>                 | <b>3</b> |
| 2.1      | Composition vs. Inheritance                         | 4        |
| <b>3</b> | <b>Polymorphism and Runtime Resolution</b>          | <b>4</b> |
| 3.1      | Method Overloading vs. Overriding                   | 4        |
| 3.2      | Dynamic Method Dispatch                             | 4        |
| 3.3      | Abstract Classes and the <code>final</code> Keyword | 5        |
| 3.4      | Access Modifiers                                    | 5        |

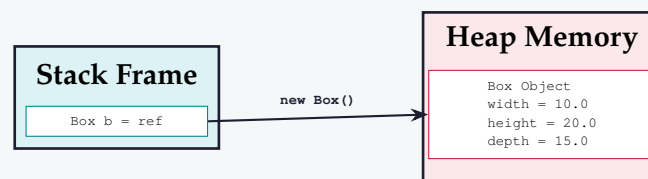
## 1. Class Fundamentals and Objects

In Java, all program logic resides within classes. A class acts as a blueprint, and objects are active physical instances of that blueprint.

### Class Declaration & Object Creation

- **Declaring a Class:** Declared using the `class` keyword. It encapsulates state (instance variables) and behavior (methods).
- **Creating Instances:** Objects are created dynamically on the heap using the `new` keyword, which allocates memory and invokes the constructor.

### Object Creation and Reference Linkage



### 1.1 Constructors in Java

A constructor initializes the state of a newly created object. It has no return type and must have the same name as the class.

- **Default Constructor:** Automatically provided by the compiler if no explicit constructor is defined. It initializes primitives to their default values (e.g., 0, false) and object references to `null`.
- **Parameterized Constructor:** User-defined constructor that accepts parameters to initialize instances with custom values.
- **Copy Constructor:** A constructor that initializes a new object using an existing object of the same class:

```
class Box {
    double width, height;
    Box(Box ob) { // Copy Constructor
        this.width = ob.width;
        this.height = ob.height;
    }
}
```

### 1.2 Argument Passing in Java

Java uses a single parameter passing mechanism: **Pass-by-Value**.

- **Primitive Arguments:** A copy of the value is passed. Changes made inside the method do not affect the original variable.

- **Object Arguments:** A copy of the reference address is passed. The method can modify the object's contents (since they both point to the same object on the heap), but reassigning the reference variable inside the method does not affect the caller's original reference.

### 1.3 The `static` Keyword

The `static` keyword is used to create class-level members that are shared across all instances.

- **Static Variables:** Single memory copy allocated in the method area, shared by all objects of that class.
- **Static Methods:** Methods that can be called without creating an instance. They can only access static data and call other static methods. They cannot use the `this` or `super` keywords.
- **Static Blocks:** Used to initialize static variables. Executed automatically when the class is first loaded into memory, before the main method or object constructors.

### 1.4 Nested and Inner Classes

Java allows defining a class inside another class.

- **Member Inner Class:** A non-static class declared inside another. It has access to all members (including private ones) of the outer class. It requires an instance of the outer class to exist.
- **Static Nested Class:** Declared static. It behaves like a top-level class packaged inside another class. It cannot access non-static outer members.
- **Local Inner Class:** Defined inside a block or method. It can only access local variables if they are effectively final.
- **Anonymous Inner Class:** An inner class defined inline without a name, typically used for overriding listener methods or implementing interfaces on the fly.

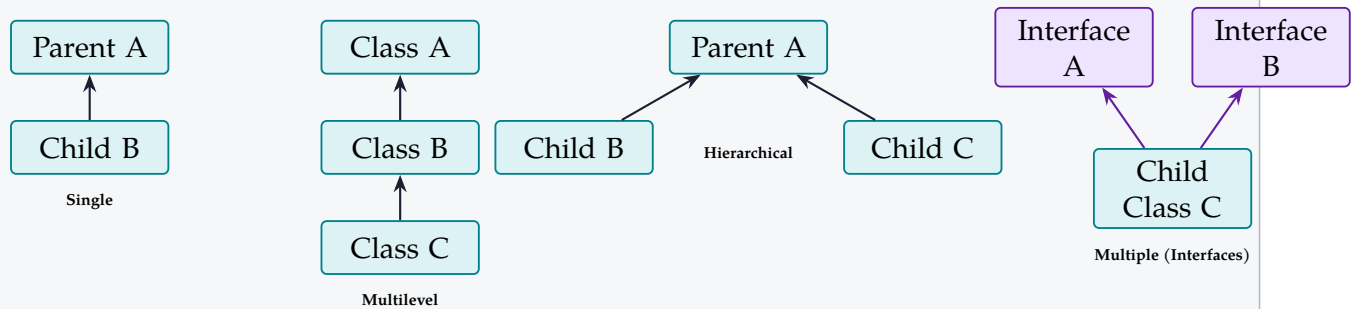
## 2. Inheritance and Reuse Models

Inheritance enables hierarchical classifications. In Java, classes are extended using the `extends` keyword.

#### Inheritance Characteristics

- **Single Inheritance Only:** Java classes can only extend **one** parent class to prevent the diamond problem (ambiguity under duplicate variable/method signatures).
- **The `super` Keyword:** Used to invoke the parent class constructor (must be the first statement in the subclass constructor), access overridden parent methods, or read hidden parent variables.

## Inheritance Trees supported in Java



## 2.1 Composition vs. Inheritance

Design patterns advocate favoring object composition over class inheritance:

| Metric                | Composition (Has-A Relationship)                 | Inheritance (Is-A Relationship)                |
|-----------------------|--------------------------------------------------|------------------------------------------------|
| <b>Coupling Level</b> | Loose coupling; internal details remain private. | Tight coupling; breaks encapsulation boundary. |
| <b>Flexibility</b>    | Behaviors can change dynamically at runtime.     | Static; behavior fixed at compilation time.    |
| <b>Design Reuse</b>   | Reuses functional components as class members.   | Reuses code by expanding a class hierarchy.    |
| <b>Example</b>        | Car has an Engine instance.                      | Car is a subclass of Vehicle.                  |

## 3. Polymorphism and Runtime Resolution

### 3.1 Method Overloading vs. Overriding

These represent static and dynamic polymorphism:

| Feature                 | Method Overloading (Static)                | Method Overriding (Dynamic)                    |
|-------------------------|--------------------------------------------|------------------------------------------------|
| <b>Class Context</b>    | Occurs within the same class definition.   | Occurs across child-parent subclass hierarchy. |
| <b>Method Signature</b> | Name must be same; parameters must differ. | Signature (parameters, returns) must match.    |
| <b>Resolution Time</b>  | Compile-time (static binding).             | Runtime (dynamic binding).                     |
| <b>Keywords</b>         | None required.                             | Assisted by @Override annotation.              |

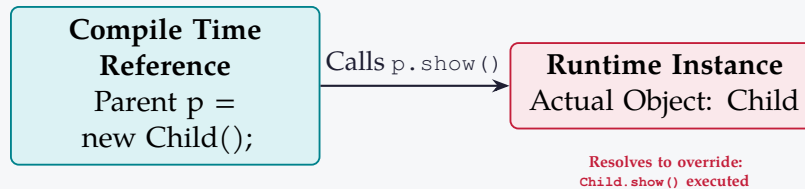
### 3.2 Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile-time.

### Dynamic Method Dispatch Rule

When an overridden method is called through a superclass reference, Java determines which version of the method to execute based on the **type of the actual object** being referred to at runtime, not the type of the reference variable.

### Dynamic Method Dispatch Flow



### 3.3 Abstract Classes and the `final` Keyword

- **Abstract Classes:** Declared using the `abstract` keyword. They cannot be instantiated. They can contain abstract methods (signatures without implementations) and concrete methods. They act as partial templates for subclasses.
- **The `final` Keyword:** Used to restrict inheritance and modifications:
  - `final` Variable: Value cannot be reassigned (behaves as a constant).
  - `final` Method: Cannot be overridden by subclasses.
  - `final` Class: Cannot be inherited/extended.

### 3.4 Access Modifiers

Java regulates member visibility using four access levels:

- **`private`:** Visible only within the declaring class.
- **Default (no modifier):** Visible only to classes inside the same package.
- **`protected`:** Visible to the same package and subclass child files in external packages.
- **`public`:** Accessible from any class anywhere in the project.

## Quick Revision — Module 3

Classes, Inheritance & Polymorphism | OE-EC704A

### Class, Constructor & Inheritance Syntax

| Concept                  | Syntax and Example Usage                                                  |
|--------------------------|---------------------------------------------------------------------------|
| Subclass Extension       | <code>class Child extends Parent { ... }</code>                           |
| Super Constructor Call   | <code>Child(int val) { super(val); ... }</code>                           |
| Copy Constructor         | <code>Box(Box b) { this.width = b.width; this.height = b.height; }</code> |
| Static Block             | <code>static { staticVar = 10; }</code>                                   |
| Polymorphism & Variables | Core Declarations & Reference Types                                       |
| Dynamic Dispatch Ref     | <code>Parent p = new Child(); p.overriddenMethod();</code>                |
| Abstract Class           | <code>abstract class Shape { abstract void draw(); }</code>               |
| Final Method             | <code>final void display() { ... } // cannot be overridden</code>         |
| Final Variable           | <code>final int MAX_SPEED = 120; // cannot be reassigned</code>           |

### One-Line Definitions

- **Class:** A user-defined template that defines the structure and behaviors of objects.
- **Object:** A physical instance of a class allocated in memory.
- **Copy Constructor:** A constructor that initializes a new object using data from an existing object of the same class.
- **Pass-by-Value:** Parameter passing where a copy of the argument value (or reference address) is passed.
- **Static Method:** A method bound to the class itself, executable without class instantiation.
- **Dynamic Method Dispatch:** Dynamic binding resolution resolving overridden method calls at runtime.
- **Abstract Class:** A class that cannot be instantiated and may contain abstract methods.
- **Inner Class:** A class declared inside the body of another enclosing class.

### Frequently Asked / Viva Questions

#### Q1. Does Java support pass-by-reference?

No, Java is strictly pass-by-value. When passing primitive variables, their values are copied. When passing object references, a copy of the heap memory address is passed.

**Frequently Asked / Viva Questions (contd.)****Q2. Can an abstract class have constructors? Why?**

Yes. Although an abstract class cannot be instantiated directly, its constructor is called during subclass instantiation via `super()` to initialize its fields.

**Q3. What is the difference between `this()` and `super()` constructor calls?**

`this()` invokes another constructor within the same class. `super()` invokes a constructor of the parent superclass. Both must be the first line of their respective constructors.

**Q4. What is the difference between method overloading and overriding?**

Overloading occurs in the same class and is resolved at compile-time (different parameters). Overriding occurs in subclasses and is resolved at runtime (same method signature).

**Q5. Why can't static methods use the `this` keyword?**

The `this` keyword refers to the current object instance. Static methods run without any object instance context, so there is no `this` instance to reference.

**Q6. What is a copy constructor and why do we need it?**

A copy constructor takes an object of the same class as a parameter to copy its fields into a new object. It provides control over whether a shallow or deep copy of the object is created.

**Q7. What are the types of inner classes in Java?**

Java inner classes are: member inner class (non-static, declared inside class), static nested class (declared static inside class), local inner class (declared inside method), and anonymous inner class (unnamed class defined inline).

**Q8. How does Dynamic Method Dispatch work?**

When an overridden method is called through a parent class reference variable, Java resolves it at runtime based on the actual object instance type being referenced, not the reference type.

**Q9. What is the difference between a shallow copy and a deep copy?**

A shallow copy copies the object's reference fields (sharing nested objects). A deep copy creates new instances of nested objects, making the new copy independent.

**Q10. Can a final method be overloaded?**

Yes. A final method cannot be overridden (redefined in a subclass), but it can be overloaded (defined with different parameters within the same class).

**Java Access Modifier Visibility Matrix**

| Modifier         | Same Class | Same Package | Subclass | Everywhere |
|------------------|------------|--------------|----------|------------|
| <b>private</b>   | Yes        | No           | No       | No         |
| <b>Default</b>   | Yes        | Yes          | No       | No         |
| <b>protected</b> | Yes        | Yes          | Yes      | No         |
| <b>public</b>    | Yes        | Yes          | Yes      | Yes        |